IERG4320/IEMS5726 — Chapter 4: Data Representation II — Cheat Sheet

Prof. YAN Wenjing, Dr. Danny Yip | Department of Information Engineering, CUHK

Part 1: Natural Language (文本处理)

文本是非结构化数据，要先转成数字向量才能给模型用。

NLP Preprocessing Pipeline (NLP预处理流程)

分词 → 小写化 → 词干提取 → 词性标注 → 向量化(BoW/TF-IDF) 或 词嵌入(Word2Vec)

Tokenization (分词)

把一整句话拆成一个个词（token）。英文按空格拆，中文没空格需要专门工具。

```
# English tokenization (NLTK)
import nltk
sent = "Antoni Gaudi was a Spanish architect."
tokens = nltk.word_tokenize(sent)
# ['Antoni','Gaudi','was','a','Spanish','architect','.']
```

```
# Chinese tokenization (jieba)
import jieba
sentence = "..."  # Chinese text
tokens = list(jieba.cut(sentence))
```

Normalization (文本规范化)

统一大小写。House/HOUSE/house当成同一个词。

Truecasing：保留专有名词大写（如Mr. Brown vs brown color）

Stemming (词干提取)

把词还原到词根。running/runs/ran → run

```
from nltk.stem.porter import PorterStemmer
stemmer = PorterStemmer()
stemmer.stem("running")  # 'run'
stemmer.stem("beautiful")  # 'beauti'
```

POS Tagging (词性标注)

标注每个词是名词(NN)、动词(VB)、形容词(JJ)等。

```
tokens = nltk.word_tokenize(
    "I wake up at seven o'clock.")
pos = nltk.pos_tag(tokens)
# [('I','PRP'),('wake','VBP'),('up','RP'),...]
```

Chunk Grammar (块语法)

用正则规则从POS标注中提取短语（如名词短语NP）。

```
grammar = "NP: {<DT>?<JJ>*<NN>}"
parser = nltk.RegexpParser(grammar)
tree = parser.parse(pos_tags)
# Extracts noun phrases like "a dangerous business"
```

Bag-of-Words (BoW) (词袋模型)

忽略语法和词序，只看每个词出不出现/出现几次。每篇文档变成一个长向量。

```
from sklearn.feature_extraction.text import (
    CountVectorizer)
docs = ["CUHK is located in Shatin",
        "CUHK has a large campus",
        "Shatin is a district in NT"]
vec = CountVectorizer()
vec.fit(docs)
print(vec.vocabulary_)
# {'cuhk':1, 'is':5, 'located':7, 'in':4, ...}
m = vec.transform(["CUHK is in Shatin"]).todense()
# matrix([[0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0]])
```

TF-IDF (词频-逆文档频率)

给词加权重：在当前文档出现多(TF高)但在其他文档出现少(IDF高)的词更重要。

$$\text{tf-idf}(w,d) = \text{tf}(w,d) \times \log\left(\frac{N}{\text{df}(w)}\right)$$

tf = 词w在文档d中出现次数；df = 包含词w的文档数；N = 总文档数

罕见词idf高，常见词(the/a/is)idf低 → 自动降低常见词权重

```
from sklearn.feature_extraction.text import (
    TfidfVectorizer)
vec = TfidfVectorizer()
vec.fit(docs)
m = vec.transform(["CUHK is in Shatin"]).todense()
# [[0. 0.5 0. 0. 0.5 0.5 0. 0. 0. 0.5 0. 0.]]
```

TF-IDF + Stemming (先提取词干再TF-IDF)

```
from nltk import word_tokenize
from nltk.stem.porter import PorterStemmer
from sklearn.feature_extraction.text import (
    TfidfVectorizer)
stemmer = PorterStemmer()
docs = ["An apple a day keeps the doctor away",
        "Eating an apple daily can improve digestion",
        "I eat an apple daily"]
new_docs = []
for sent in docs:
    tokens = word_tokenize(sent)
    stemmed = " ".join(
        [stemmer.stem(w) for w in tokens])
    new_docs.append(stemmed)
vec = TfidfVectorizer()
vec.fit(new_docs)
m = vec.transform(new_docs).todense()
```

"keeps"和"keep"、"eating"和"eat"会被归为同一个词根

Word Embeddings (词嵌入)

把每个词映射成一个稠密向量（100–300维），语义相近的词向量也相近。

- **Word2Vec**: Skip-gram (用中心词预测周围词) / CBoW (用周围词预测中心词)
- **GloVe**: global word co-occurrence (全局共现统计)
- **fastText**: subword-level (处理子词，能应对新词)

有趣特性：king − man + woman ≈ queen

```
# Word2Vec with gensim (pre-trained)
from gensim.models.keyedvectors import (
    load_word2vec_format)
model = load_word2vec_format(
    'GoogleNews-vectors-negative300.bin',
    binary=True)
model.similarity("big","huge")  # 0.78
model["great"]  # 300-d vector
model.most_similar(["apple","orange"])
# [('peach',0.62),('pear',0.60),...]
```

Classic NLP: Pros & Cons

优点：简单可解释，适合传统ML基线

缺点：忽略语义和上下文，高维稀疏，不处理同义词/多义词

Part 2: Image Data (图像处理)

图像 = 像素矩阵。RGB图：每像素3个值(0–255)。300×300 RGB = 270,000个值。

Edge Detection (边缘检测)

找图像中亮度急剧变化的地方（物体轮廓）。

```
import cv2
img = cv2.imread('tiger.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (3,3), 0)
# Sobel edge detection
sobel = cv2.Sobel(src=blur, ddepth=cv2.CV_64F,
    dx=1, dy=1, ksize=5)
# Canny edge detection
edges = cv2.Canny(image=blur,
    threshold1=100, threshold2=200)
cv2.imwrite('sobel.png', sobel)
cv2.imwrite('canny.png', edges)
```

SIFT (尺度不变特征变换)

自动检测图像中的关键点（角点/斑点），不受缩放和旋转影响。每个关键点用128维向量描述。

4 Steps: Scale-space extrema → Keypoint localization → Orientation → 128-d descriptor

```
import cv2
img = cv2.imread('photo.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT.create()
kp, des = sift.detectAndCompute(gray, None)
# kp: keypoints, des: descriptors (Nx128)
flag = cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
img = cv2.drawKeypoints(gray, kp, img, flags=flag)
cv2.imwrite('keypoints.png', img)
```

HOG Descriptor (方向梯度直方图)

统计图像局部区域的梯度方向分布，常用于行人检测。

```
from skimage.feature import hog
from skimage.transform import resize
from skimage.io import imread
img = imread('photo.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
resized = resize(gray, (128*4, 64*4))
fd, hog_img = hog(resized, orientations=9,
    pixels_per_cell=(8,8), cells_per_block=(2,2),
    visualize=True)
```

Visual Bag-of-Words (视觉词袋)

类比文本BoW：SIFT提取关键点 → K-means聚类成"视觉词典" → 每张图变成直方图向量。

局限：SIFT关键点较原始，不含高层语义，计算慢。

Image Tensors (图像张量)

深度学习中图像表示为多维数组。PyTorch格式：(N, C, H, W)，TensorFlow：(N, H, W, C)。

```
import torch
from PIL import Image
from torchvision import transforms
img = Image.open("cat.jpg")
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),  # -> (C,H,W)
    transforms.Normalize(
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225])  # ImageNet
])
tensor = transform(img)  # (3, 224, 224)
batch = tensor.unsqueeze(0)  # (1, 3, 224, 224)
```

常见操作：

```
# Reshape: (N,H,W,C) -> (N,C,H,W)
t = t.permute(0, 3, 1, 2)
# Crop a region
cropped = tensor[:, :, 50:150, 50:150]
```

Part 3: Audio Data (音频处理)

音频 = 声波的数字采样。关键参数：采样率(sr)、通道数、采样位宽。

Duration = frames / sample_rate

Read/Write Audio (读写音频)

```
import soundfile as sf
y, sr = sf.read('audio.wav')  # y: numpy array
print(y.shape, sr)  # (frames, channels), Hz
sf.write('out.wav', y, sr)
```

FFmpeg: Trim & Convert (裁剪转换)

```
import ffmpeg
ffmpeg.input("song.mp3",
    ss='00:00:10', to='00:00:20') \
    .output("clip.wav").run()
```

Pitch (音高)

声音频率，Hz越高音越高。A4=440Hz。

```
import librosa, numpy as np
y, sr = librosa.load("song.mp3", sr=44100)
f0, voiced, probs = librosa.pyin(y,
    fmin=librosa.note_to_hz('C2'),
    fmax=librosa.note_to_hz('C7'))
mean_pitch = np.nanmean(f0)
```

注意：librosa目前只支持Python $\leq$ 3.12，需要conda虚拟环境

Timbre Features (音色特征)

音色让你区分钢琴和吉他弹同一个音。用MFCC、频谱对比度、色度特征描述。

```
mfccs = librosa.feature.mfcc(
    y=y, sr=sr, n_mfcc=13)
spectral_contrast = \
    librosa.feature.spectral_contrast(y=y,sr=sr)
chroma = librosa.feature.chroma_stft(y=y,sr=sr)
```

Audio-to-Tensor Pipeline (音频转张量流水线)

深度学习需要统一格式，完整流程如下：

读取 → 统一通道(mono/stereo) → 统一采样率 → 统一长度(pad/truncate) → 数据增强(time shift) → Mel频谱图 → 数据增强(masking)

```
import torchaudio
# 1. Read audio
sig, sr = torchaudio.load("audio.wav")
```

```
# 2. Mono <-> Stereo
def rechannel(aud, new_ch):
    sig, sr = aud
    if sig.shape[0] == new_ch: return aud
    if new_ch == 1:
        resig = sig[:1, :]  # keep 1st channel
    else:
        resig = torch.cat([sig, sig])  # dup
    return (resig, sr)
```

```
# 3. Resample to target rate
def resample(aud, newsr):
    sig, sr = aud
    if sr == newsr: return aud
    resig = torchaudio.transforms.Resample(
        sr, newsr)(sig[:1,:])
    if sig.shape[0] > 1:  # stereo
        retwo = torchaudio.transforms.Resample(
            sr, newsr)(sig[1:,:])
        resig = torch.cat([resig, retwo])
    return (resig, newsr)
```

```
# 4. Pad short / truncate long
def pad_trunc(aud, max_ms):
    sig, sr = aud
    num_rows, sig_len = sig.shape
    max_len = sr // 1000 * max_ms
    if sig_len > max_len:
        sig = sig[:, :max_len]
    elif sig_len < max_len:
        pad_b = random.randint(0,max_len-sig_len)
        pad_e = max_len - sig_len - pad_b
        sig = torch.cat((torch.zeros((num_rows,
            pad_b)), sig, torch.zeros((num_rows,
            pad_e))), 1)
    return (sig, sr)
```

```
# 5. Time shift (data augmentation)
def time_shift(aud, shift_limit):
    sig, sr = aud
    _, sig_len = sig.shape
    shift = int(random.random()*shift_limit*sig_len)
    return (sig.roll(shift), sr)
```

Mel Spectrogram (梅尔频谱图)

把音频转成"图片"：x轴=时间，y轴=梅尔频率，颜色=分贝强度。是音频深度学习的标准输入。

```
# 6. Generate Mel Spectrogram
from torchaudio import transforms
def spectro_gram(aud, n_mels=64,
                 n_fft=1024, hop_len=None):
    sig, sr = aud
    spec = transforms.MelSpectrogram(
        sr, n_fft=n_fft, hop_length=hop_len,
        n_mels=n_mels)(sig)
    spec = transforms.AmplitudeToDB(
        top_db=80)(spec)
    return spec  # (channel, n_mels, time)
```

Time/Freq Masking (时频遮罩增强)

随机遮住频谱图的一部分时间段/频率段，防止过拟合。

```
# 7. Spectrogram augmentation
def spectro_augment(spec, max_mask_pct=0.1,
                    n_freq_masks=1, n_time_masks=1):
    _, n_mels, n_steps = spec.shape
    mask_value = spec.mean()
    aug = spec
    freq_param = max_mask_pct * n_mels
    for _ in range(n_freq_masks):
        aug = transforms.FrequencyMasking(
            freq_param)(aug, mask_value)
    time_param = max_mask_pct * n_steps
    for _ in range(n_time_masks):
        aug = transforms.TimeMasking(
            time_param)(aug, mask_value)
    return aug
```

Quick Load MP3 to Tensor

```
import torchaudio
def mp3_to_tensor(path, target_sr=16000):
    waveform, sr = torchaudio.load(
        path, format="mp3")
    if sr != target_sr:
        waveform = torchaudio.transforms \
            .Resample(sr, target_sr)(waveform)
    return waveform, target_sr
```

Quick Reference (速查总结)

Data	Method	Key Tool
4*Text	Tokenize	nltk / jieba
	Stem	PorterStemmer
	BoW / TF-IDF	CountVec / TfidfVec
	Embedding	gensim Word2Vec
4*Image	Edge detect	cv2.Sobel / Canny
	Keypoints	cv2.SIFT.create()
	HOG	skimage.feature.hog
	Tensor	torchvision.transforms
4*Audio	Read/Write	soundfile / torchaudio
	Pitch	librosa.pyin
	Mel Spec	MelSpectrogram
	Masking	Freq/TimeMasking

核心思路：不管什么数据(文本/图像/音频)，最终都要转成数字向量/张量才能给模型用。